

PYTHON PROJECT BUILDING GUIDE

A Progressive, Milestone-Based Hands-On Tutorial for Beginners

Building a CLI Calculator & Airline Flight Management System From Scratch

Target Level: Absolute Beginner

Version Control: Git Workflow Included

Style: Step-by-Step Practical

1. Course Overview & Curriculum Constraints

Welcome to your project-based Python journey! Learning syntax alone won't make you a developer — the true test of programming mastery lies in constructing functional applications from scratch. In this guide, you will build two complete, real-world CLI (Command Line Interface) applications step by step using strict version control practices with Git.

ALLOWED PYTHON CONCEPTS (What You Know Already)

This guide is crafted exclusively using foundational Python building blocks that you have already learned. You are fully equipped to complete both projects using only:

- **Variables:** Storing values in memory.
- **Basic Data Types:** Strings (`str`), Integers (`int`), Floats (`float`), Booleans (`bool`).
- **Input & Output:** `input()` for receiving text and `print()` for displaying formatted messages.
- **Operators:** Arithmetic (`+`, `-`, `*`, `/`, `%`, `**`), Comparison (`==`, `!=`, `>`, `<`, `>=`, `<=`), Logical (`and`, `or`, `not`).
- **Control Flow:** Conditional statements (`if`, `elif`, `else`), Pattern matching (`match / case`).
- **Loops:** Iteration using `for` loops, state retention using `while` loops, control using `break` and `continue`.
- **Functions:** Defining subroutines (`def`), returning values (`return`), passing positional arguments.
- **Lists:** Basic 1D and nested 2D list collection structures (indexed data storage).

STRICTLY RESTRICTED CONCEPTS (What Is NOT Used)

To ensure this guide stays completely aligned with your current learning stage, the following advanced topics are **100% EXCLUDED** from the project implementations:

No Dictionaries, No Classes / OOP, No Objects, No File Handling / I/O, No JSON, No Databases / SQL, No APIs, No Exception Handling (`try/except`), No Decorators, No Lambda functions, No List Comprehensions, No `*args` / `**kwargs`, and No external modules.

2. Version Control & Git Workflow

Professional developers never work without version control. Git tracks changes to your project, allowing you to review history, revert mistakes, and organize your work into logical milestones.

ESSENTIAL GIT COMMANDS REFERENCE

Git Command	What It Does	When To Use It
<code>git init</code>	Initializes a new Git repository in the current folder.	Once at the start of a new project directory.
<code>git status</code>	Shows modified, staged, or untracked files.	Before staging or committing to inspect project state.
<code>git add .</code>	Stages all changed and new files for the next commit.	When a milestone feature is finished and tested.
<code>git commit -m "msg"</code>	Saves a permanent snapshot of staged changes with a descriptive message.	At the end of every completed milestone.
<code>git log --oneline</code>	Displays a concise timeline of all past commits.	To review commit history and verify progress.

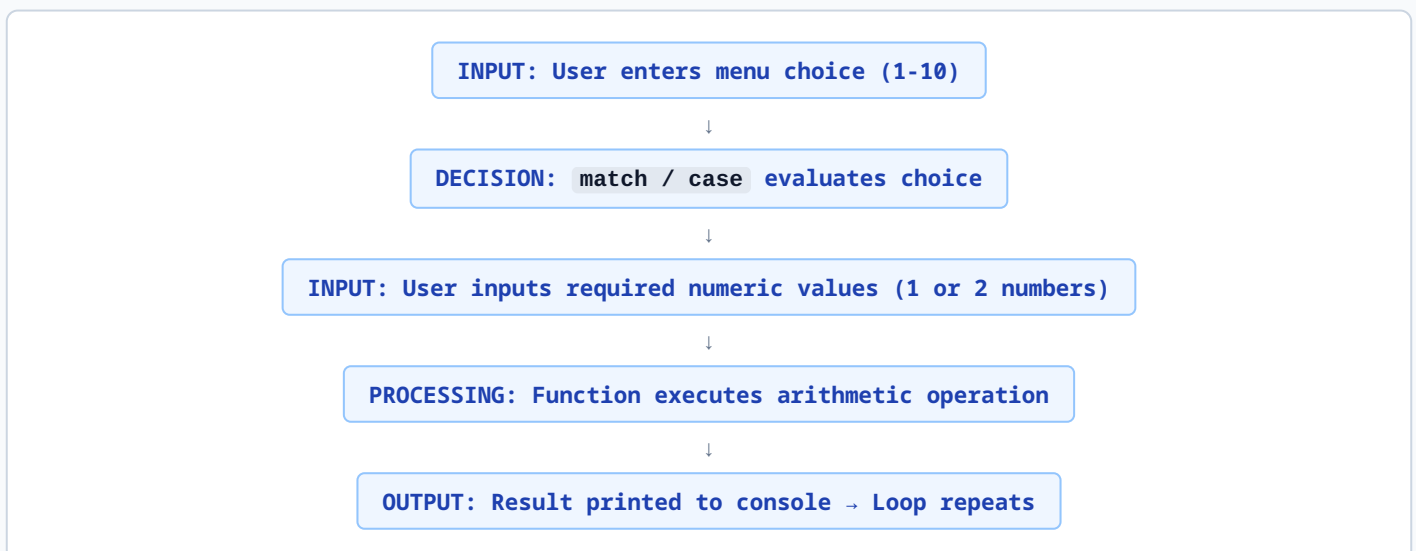
GOLDEN RULE OF COMMITTING: Never wait until the entire program is finished to commit. Make small, atomic commits every time a milestone feature is complete and verified working.

3. PROJECT 1 — CLI CALCULATOR

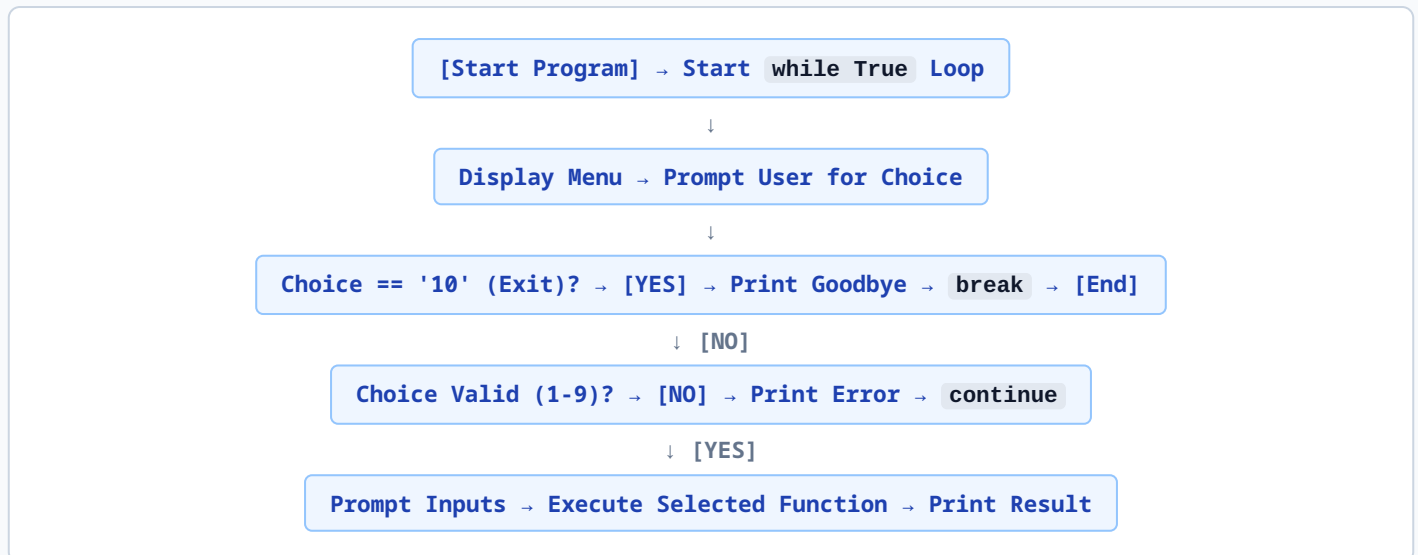
Project Objective & Architecture

The goal of Project 1 is to build an interactive Command-Line Interface (CLI) Calculator. The calculator runs continuously in a terminal, offering a clear menu of 10 options (Addition, Subtraction, Multiplication, Division, Modulus, Power, Average, Maximum, Minimum, and Exit).

CORE PROGRAM ARCHITECTURE: INPUT → PROCESSING → DECISION → OUTPUT



MAIN LOOP FLOWCHART



MILESTONE 1 — Calculator Setup & Project Structure

What you know already: Terminal commands, basic file system navigation.

What you are learning now: Directory setup, initializing a Git repo, creating the entry Python file.

What you should be able to do after this: Set up a clean environment with version control.

IMPLEMENTATION STEPS

1. Open your terminal and create a folder named `cli_calculator`.
2. Navigate into the folder and create `calculator.py` and a `README.md` file.
3. Initialize Git and run your first test script.

In `calculator.py`, write:

```
# CLI Calculator - Project 1
# Author: Beginner Student

print("=====")
print("    PYTHON CLI CALCULATOR    ")
print("=====")
print("Calculator initialized successfully!")
```

Milestone 1 Completion Checklist:

- ☐ Project directory and files created
- ☐ Program executed successfully using `python3 calculator.py`
- ☐ Git repository initialized
- ☐ Code fully understood

COMMIT BEFORE CONTINUING:

```
git status
git add .
git commit -m "Milestone 1: Set up calculator project structure"
```

MILESTONE 2 — Interactive Menu & Main Control Loop

What you know already: `print()`, `input()`, strings.

What you are learning now: Endless `while True` loop, structural pattern matching with `match / case`, terminating with `break`.

What you should be able to do after this: Create an interactive menu loop that processes choices and exits on command.

CODE IMPLEMENTATION

```
while True:
    print("
=====")
    print("          PYTHON CALCULATOR          ")
    print("=====")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Exit")
    print("=====")

    choice = input("Enter choice (1-3): ")

    match choice:
        case "1":
            print("You selected Addition.")
        case "2":
            print("You selected Subtraction.")
        case "3":
            print("Thank you for using Python Calculator. Goodbye!")
            break
        case _:
            print("Invalid choice! Please select a valid option from the menu.")
```

LINE-BY-LINE CODE EXPLANATION

- `while True:` Creates an infinite loop so the calculator stays open after every operation.
- `input("Enter choice (1-3): ")` Captures the user's input as a string.
- `match choice:` Evaluates the string stored in `choice`.
- `case "1":` Executed if the user types `1`.
- `case _:` The wildcard case. Executes if input matches none of the specified cases.
- `break` Breaks out of the loop and terminates the program when option `3` is chosen.

Milestone 2 Completion Checklist:

- ☐ Menu displays correctly in terminal
- ☐ Valid options print placeholder message
- ☐ Invalid inputs trigger default case message
- ☐ Option 3 terminates loop cleanly

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 2: Implement main menu loop with match/case"
```

MILESTONE 3 — Basic Arithmetic (Addition & Subtraction)

What you know already: Basic arithmetic operators (+, -), float conversion.

What you are learning now: Function definition (def), positional arguments, returning calculated results.

What you should be able to do after this: Modularize math logic into functions and integrate them with user input.

CODE IMPLEMENTATION

```
# Define Addition and Subtraction functions
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# Inside the main while loop match/case block:
match choice:
    case "1":
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = add(num1, num2)
        print("Result: " + str(num1) + " + " + str(num2) + " = " + str(result))
    case "2":
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = subtract(num1, num2)
        print("Result: " + str(num1) + " - " + str(num2) + " = " + str(result))
```

LINE-BY-LINE CODE EXPLANATION

- `def add(a, b):` Defines a function named `add` taking two positional arguments.
- `return a + b` Computes the sum and sends the result back to the caller.
- `float(input(...))` Converts user input string into a float number to allow decimals.
- `result = add(num1, num2)` Calls the function passing `num1` and `num2` as arguments.

Milestone 3 Completion Checklist:

- ☐ Functions `add()` and `subtract()` defined
- ☐ Float inputs collected cleanly
- ☐ Calculations accurate and results printed

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 3: Add addition and subtraction functions"
```

MILESTONE 4 — Multiplication & Safe Division

What you know already: Arithmetic operators (*, /), if/else conditionals.

What you are learning now: Safe division without exception handling (preventing zero division using if checks).

What you should be able to do after this: Prevent mathematical program crashes using logical checks.

CODE IMPLEMENTATION

```
def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero is undefined!"
    return a / b

# Menu Integration
match choice:
    case "3": # Multiplication
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        print("Result: " + str(multiply(num1, num2)))
    case "4": # Division
        num1 = float(input("Enter dividend: "))
        num2 = float(input("Enter divisor: "))
        print("Result: " + str(divide(num1, num2)))
```

TEACHING CONCEPT: ZERO DIVISION CHECK

Attempting `10 / 0` in Python causes a fatal crash (`ZeroDivisionError`). Because exception handling is restricted, we use conditional evaluation: `if b == 0: return "Error..."`. If `b` is zero, the function immediately returns an error string, bypassing the division line entirely!

Milestone 4 Completion Checklist:

- ☐ Multiplication verified working
- ☐ Division with valid non-zero numbers tested
- ☐ Division by zero tested (returns error message, no crash)

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 4: Implement multiplication and safe division"
```

MILESTONE 5 — Modulus & Power Operations

What you know already: Advanced arithmetic operators (`%`, `**`).

What you are learning now: Exponentiation and remainder calculations.

What you should be able to do after this: Add mathematical modularity and power features.

CODE IMPLEMENTATION

```
def modulus(a, b):
    if b == 0:
        return "Error: Modulus by zero is undefined!"
    return a % b

def power(base, exponent):
    return base ** exponent
```

Milestone 5 Completion Checklist:

- ☐ Modulus function prevents zero division
- ☐ Power function calculates base^exponent correctly

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 5: Add modulus and power functions"
```

MILESTONE 6 — Sequence Math (Average, Max, Min using Lists)

What you know already: `for` loops, lists, accumulator pattern.

What you are learning now: Processing series of numbers without built-in `sum()`, `max()`, or `min()`.

What you should be able to do after this: Calculate aggregate data over sequence inputs.

CODE IMPLEMENTATION & EXPLANATIONS

```
def calculate_average(numbers):
    if len(numbers) == 0:
        return 0
    total = 0.0
    for num in numbers:
        total = total + num
    return total / len(numbers)

def find_maximum(numbers):
    if len(numbers) == 0:
        return None
    max_val = numbers[0]
    for num in numbers:
        if num > max_val:
            max_val = num
    return max_val

def find_minimum(numbers):
    if len(numbers) == 0:
        return None
    min_val = numbers[0]
    for num in numbers:
        if num < min_val:
            min_val = num
    return min_val
```

LINE-BY-LINE LOGIC

- `max_val = numbers[0]` : We initialize our benchmark with the first item in the list.
- `for num in numbers: if num > max_val: max_val = num` : We iterate through every number. If we find a number larger than our current benchmark, we update `max_val` !

Milestone 6 Completion Checklist:

- ☐ Average algorithm tested with sample series
- ☐ Maximum and Minimum logic verified manually

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 6: Add average, maximum, and minimum list functions"
```

MILESTONE 7 — UI Polish, Input Helper & Final Calculator Integration

What you know already: List accumulation, loop control.

What you are learning now: Helper functions for sequence input collection and user experience polish.

What you should be able to do after this: Finalize the CLI Calculator program.

HELPER FUNCTION FOR COLLECTING SEQUENCES

```
def get_numbers_input():
    count = int(input("How many numbers do you want to enter? "))
    numbers = []
    for i in range(count):
        val = float(input("Enter number " + str(i + 1) + ": "))
        numbers.append(val)
    return numbers
```

Milestone 7 Completion Checklist:

- ☐ All 10 options integrated seamlessly in menu
- ☐ Full end-to-end user workflow tested

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 7: Polish UI and complete Calculator project"
```

4. PROJECT 2 — AIRLINE FLIGHT MANAGEMENT SYSTEM

Project Objective & Flight Data Structure

In Project 2, you will build a Flight Management System for a regional airline. The program allows users to view available flights, search by destination, calculate pricing, book seats, and cancel bookings.

UNDERSTANDING NESTED LISTS (2D LISTS) FOR FLIGHT DATA

Since OOP classes and dictionaries are restricted, we store tabular flight data using a **2D List** (a list of lists):

```
flights = [
    ["KQ101", "Nairobi", "Mombasa", 8000],
    ["KQ102", "Nairobi", "Kisumu", 5000],
    ["KQ103", "Nairobi", "Eldoret", 6000],
    ["KQ104", "Nairobi", "Malindi", 9000]
]
```

VISUALIZING THE LIST INDEXING

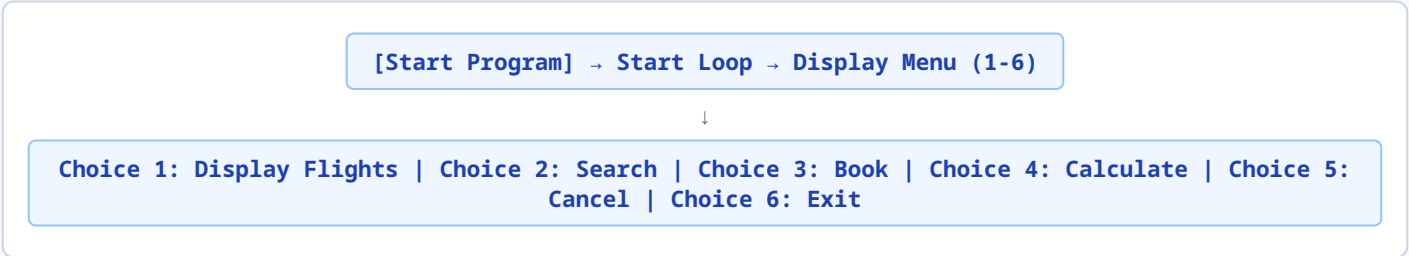
List Element	Index 0 (Flight Code)	Index 1 (Origin)	Index 2 (Destination)	Index 3 (Price KSh)
flights[0]	"KQ101" (flights[0][0])	"Nairobi" (flights[0][1])	"Mombasa" (flights[0][2])	8000 (flights[0][3])
flights[1]	"KQ102" (flights[1][0])	"Nairobi" (flights[1][1])	"Kisumu" (flights[1][2])	5000 (flights[1][3])

HOW ITERATION WORKS

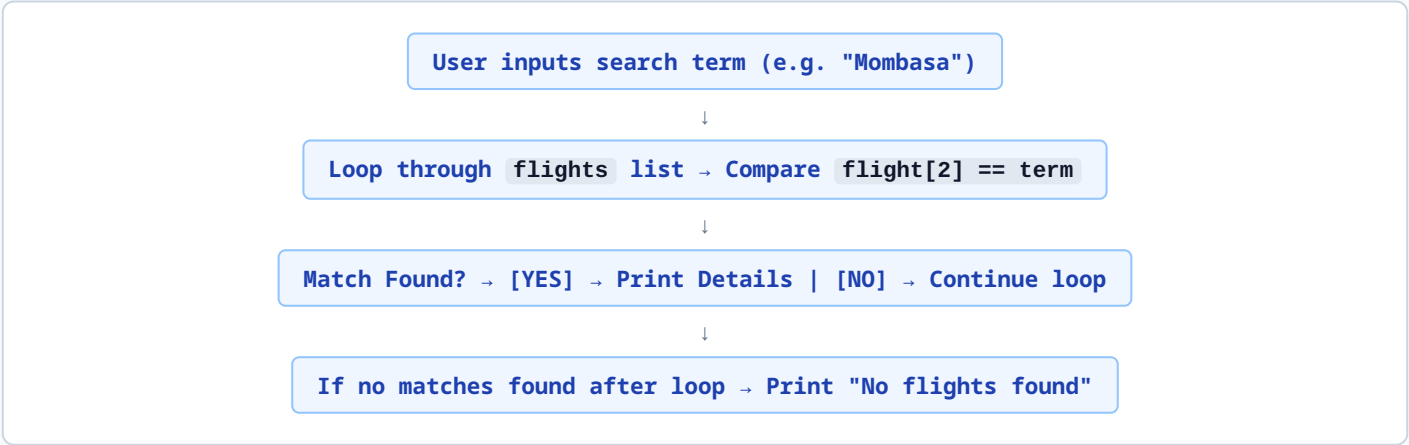
```
for flight in flights:
    print("Flight Code: " + flight[0] + " | Destination: " + flight[2] + " | Price: KSh " +
          str(flight[3]))
```

AIRLINE SYSTEM FLOWCHARTS

Main Menu Loop:



Flight Search Flow:



MILESTONE 1 — Airline Setup & Repository Creation

What you know already: Shell navigation, Git setup.

What you are learning now: Initializing the airline project directory.

What you should be able to do after this: Create repository structure for Project 2.

Milestone 1 Completion Checklist:

- ☐ Directory `airline_system` created
- ☐ `airline.py` initialized with greeting header

☐ Git repo created

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 1: Initialize airline management system project"
```

MILESTONE 2 — Flight Data Structure & Display Subroutine

What you know already: 2D Lists, `for` loops.

What you are learning now: Formatting list elements into readable flight tables.

What you should be able to do after this: Print formatted schedules from list data.

CODE IMPLEMENTATION

```
flights = [
    ["KQ101", "Nairobi", "Mombasa", 8000],
    ["KQ102", "Nairobi", "Kisumu", 5000],
    ["KQ103", "Nairobi", "Eldoret", 6000],
    ["KQ104", "Nairobi", "Malindi", 9000]
]

def display_flights(flight_list):
    print("
    -----")
    print("CODE      | ORIGIN      | DESTINATION | PRICE (KSh)")
    print("-----")
    for flight in flight_list:
        code = flight[0]
        origin = flight[1]
        dest = flight[2]
        price = str(flight[3])
        print(code + " | " + origin + " | " + dest + " | KSh " + price)
    print("-----")
```

Milestone 2 Completion Checklist:

- ☐ 2D list structure defined
- ☐ `display_flights()` prints formatted table

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 2: Define flight data structure and display function"
```

MILESTONE 3 — Search Functionality

What you know already: String operations, `for` loops, flag variables.

What you are learning now: Searching nested collections with flag variables.

What you should be able to do after this: Filter lists based on target search criteria.

CODE IMPLEMENTATION

```
def search_flight(flight_list, destination):
    found = False
    print("
Search Results for: " + destination)
    print("-----")
    for flight in flight_list:
        # Case-insensitive comparison using lower()
        if flight[2].lower() == destination.lower():
            print("FOUND: " + flight[0] + " | " + flight[1] + " -> " + flight[2] + " | Price:
KSh " + str(flight[3]))
            found = True
    if not found:
        print("No flights found matching destination: " + destination)
    print("-----")
```

Milestone 3 Completion Checklist:

- ☐ Search matches valid destination correctly
- ☐ Search handles invalid destinations gracefully without errors

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 3: Implement flight search feature"
```

MILESTONE 4 — Ticket Pricing & Multi-Passenger Calculations

What you know already: Arithmetic multiplication, positional parameters.

What you are learning now: Calculating total fare based on passenger counts.

What you should be able to do after this: Compute costs dynamically.

CODE IMPLEMENTATION

```
def calculate_price(unit_price, passenger_count):
    total = unit_price * passenger_count
    # Group discount: 10% off for 5 or more passengers
    if passenger_count >= 5:
        discount = total * 0.10
        total = total - discount
        print("Group discount (10%) applied!")
    return total
```

Milestone 4 Completion Checklist:

- ☐ Ticket cost calculates accurately for single & multiple tickets
- ☐ Group discount triggers correctly for 5+ passengers

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 4: Add ticket pricing and discount calculator"
```

MILESTONE 5 — Booking & Seat Reservation System

What you know already: Modifying list elements, state tracking in lists.

What you are learning now: Managing bookings using parallel lists.

What you should be able to do after this: Append and update records in list structures.

CODE IMPLEMENTATION

```
# Track bookings in a list: [booking_id, flight_code, passenger_name, passengers, total_cost]
bookings = []
booking_counter = 1001

def book_flight(flight_list, bookings_list, next_id):
    flight_code = input("Enter Flight Code to book (e.g. KQ101): ")

    selected_flight = None
    for flight in flight_list:
        if flight[0].lower() == flight_code.lower():
            selected_flight = flight
            break

    if selected_flight is None:
        print("Error: Flight code not found!")
        return next_id

    passenger_name = input("Enter Passenger Full Name: ")
    num_passengers = int(input("Enter Number of Passengers: "))

    total_price = calculate_price(selected_flight[3], num_passengers)

    # Store booking record
    new_booking = [next_id, selected_flight[0], passenger_name, num_passengers, total_price]
    bookings_list.append(new_booking)

    print("
BOOKING CONFIRMED!")
    print("Booking Reference: BK-" + str(next_id))
    print("Passenger: " + passenger_name)
    print("Flight: " + selected_flight[0] + " (" + selected_flight[1] + " -> " +
selected_flight[2] + ")")
    print("Total Fare: KSh " + str(total_price))

    return next_id + 1
```

Milestone 5 Completion Checklist:

- ☐ Flight code lookup verified
- ☐ Booking created and stored in `bookings` list
- ☐ Reference ID increments for next customer

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 5: Implement flight booking workflow"
```

MILESTONE 6 — Cancellation Process & Reservation Management

What you know already: List removal, iteration.

What you are learning now: Locating and deleting items from lists safely.

What you should be able to do after this: Cancel and remove active bookings.

CODE IMPLEMENTATION

```
def cancel_booking(bookings_list):
    ref_id = int(input("Enter Booking Reference ID (numbers only, e.g. 1001): "))

    found_index = -1
    for i in range(len(bookings_list)):
        if bookings_list[i][0] == ref_id:
            found_index = i
            break

    if found_index != -1:
        cancelled = bookings_list.pop(found_index)
        print("Booking BK-" + str(ref_id) + " for " + cancelled[2] + " has been successfully cancelled.")
    else:
        print("Error: Booking Reference ID not found.")
```

Milestone 6 Completion Checklist:

- ☐ Valid booking reference removed from list using `pop()`
- ☐ Invalid booking reference handles error safely

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 6: Add booking cancellation feature"
```

MILESTONE 7 — Interactive CLI Menu & System Integration

What you know already: `match / case`, endless loops.

What you are learning now: Wire all functions into a cohesive application.

What you should be able to do after this: Complete the Airline Management System.

Milestone 7 Completion Checklist:

- ☐ All 6 menu actions hooked up correctly
- ☐ Tested full booking, viewing, and cancellation flow

COMMIT BEFORE CONTINUING:

```
git add .
git commit -m "Milestone 7: Assemble complete Airline Management System"
```

5. Reference Source Code & Appendices

Complete Project 1 Code: CLI Calculator

```
# =====
# PROJECT 1: COMPLETE CLI CALCULATOR
# Restrictive Concepts Applied: No OOP, No Exception Handling, No Modules
# =====

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero is undefined!"
    return a / b

def modulus(a, b):
    if b == 0:
        return "Error: Modulus by zero is undefined!"
    return a % b

def power(a, b):
    return a ** b

def calculate_average(numbers):
    if len(numbers) == 0:
        return 0
    total = 0.0
    for num in numbers:
        total += num
    return total / len(numbers)

def find_maximum(numbers):
    if len(numbers) == 0:
        return None
    max_val = numbers[0]
    for num in numbers:
        if num > max_val:
            max_val = num
    return max_val

def find_minimum(numbers):
    if len(numbers) == 0:
        return None
    min_val = numbers[0]
    for num in numbers:
        if num < min_val:
            min_val = num
    return min_val

def get_numbers_input():
    count = int(input("How many numbers do you want to enter? "))
    numbers = []
```

```

for i in range(count):
    val = float(input("Enter number " + str(i + 1) + ": "))
    numbers.append(val)
return numbers

def main_calculator():
    while True:
        print("
=====")
        print("          PYTHON CALCULATOR          ")
        print("=====")
        print("1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")
        print("5. Modulus")
        print("6. Power")
        print("7. Average")
        print("8. Maximum")
        print("9. Minimum")
        print("10. Exit")
        print("=====")

        choice = input("Enter choice (1-10): ")

        match choice:
            case "1":
                a = float(input("Enter first number: "))
                b = float(input("Enter second number: "))
                print("Result: " + str(add(a, b)))
            case "2":
                a = float(input("Enter first number: "))
                b = float(input("Enter second number: "))
                print("Result: " + str(subtract(a, b)))
            case "3":
                a = float(input("Enter first number: "))
                b = float(input("Enter second number: "))
                print("Result: " + str(multiply(a, b)))
            case "4":
                a = float(input("Enter dividend: "))
                b = float(input("Enter divisor: "))
                print("Result: " + str(divide(a, b)))
            case "5":
                a = float(input("Enter dividend: "))
                b = float(input("Enter divisor: "))
                print("Result: " + str(modulus(a, b)))
            case "6":
                a = float(input("Enter base: "))
                b = float(input("Enter exponent: "))
                print("Result: " + str(power(a, b)))
            case "7":
                nums = get_numbers_input()
                print("Average: " + str(average(nums)))
            case "8":
                nums = get_numbers_input()
                print("Maximum: " + str(find_maximum(nums)))
            case "9":
                nums = get_numbers_input()
                print("Minimum: " + str(find_minimum(nums)))
            case "10":
                print("Thank you for using Python Calculator. Goodbye!")
                break

```

```

        case _:
            print("Invalid choice! Please select 1-10.")

if __name__ == "__main__":
    main_calculator()

```

Complete Project 2 Code: Airline Management System

```

# =====
# PROJECT 2: COMPLETE AIRLINE MANAGEMENT SYSTEM
# Restrictive Concepts Applied: Lists only, No Dictionaries, No Classes
# =====

flights = [
    ["KQ101", "Nairobi", "Mombasa", 8000],
    ["KQ102", "Nairobi", "Kisumu", 5000],
    ["KQ103", "Nairobi", "Eldoret", 6000],
    ["KQ104", "Nairobi", "Malindi", 9000]
]

bookings = []
next_booking_id = 1001

def display_flights(flight_list):
    print("
    -----")
    print("CODE      | ORIGIN        | DESTINATION  | PRICE (KSh)")
    print("-----")
    for flight in flight_list:
        print(flight[0] + " | " + flight[1] + " | " + flight[2] + " | KSh " +
str(flight[3]))
    print("-----")

def search_flight(flight_list, destination):
    found = False
    print("
    Search Results for: " + destination)
    print("-----")
    for flight in flight_list:
        if flight[2].lower() == destination.lower():
            print("FOUND: " + flight[0] + " | " + flight[1] + " -> " + flight[2] + " | Price: KSh "
+ str(flight[3]))
            found = True
    if not found:
        print("No flights found matching destination: " + destination)
    print("-----")

def calculate_price(unit_price, passenger_count):
    total = unit_price * passenger_count
    if passenger_count >= 5:
        total -= (total * 0.10)
        print("Group discount (10%) applied!")
    return total

def book_flight(flight_list, bookings_list, next_id):
    flight_code = input("Enter Flight Code to book (e.g. KQ101): ")
    selected_flight = None
    for flight in flight_list:
        if flight[0].lower() == flight_code.lower():
            selected_flight = flight
            break

```

```

if selected_flight is None:
    print("Error: Flight code not found!")
    return next_id

passenger_name = input("Enter Passenger Full Name: ")
num_passengers = int(input("Enter Number of Passengers: "))

total_price = calculate_price(selected_flight[3], num_passengers)

new_booking = [next_id, selected_flight[0], passenger_name, num_passengers, total_price]
bookings_list.append(new_booking)

print("
BOOKING CONFIRMED!")
print("Booking Reference: BK-" + str(next_id))
print("Passenger: " + passenger_name)
print("Flight: " + selected_flight[0] + " (" + selected_flight[1] + " -> " + selected_flight[2]
+ ")")
print("Total Fare: KSh " + str(total_price))

return next_id + 1

def cancel_booking(bookings_list):
    ref_input = input("Enter Booking Reference ID (e.g. 1001): ")
    ref_id = int(ref_input)

    found_index = -1
    for i in range(len(bookings_list)):
        if bookings_list[i][0] == ref_id:
            found_index = i
            break

    if found_index != -1:
        cancelled = bookings_list.pop(found_index)
        print("Booking BK-" + str(ref_id) + " for " + cancelled[2] + " has been successfully
cancelled.")
    else:
        print("Error: Booking Reference ID not found.")

def main_airline():
    global next_booking_id
    while True:
        print("
=====")
        print("          AIRLINE MANAGEMENT SYSTEM          ")
        print("=====")
        print("1. View available flights")
        print("2. Search for a flight")
        print("3. Book a flight")
        print("4. Calculate ticket price")
        print("5. Cancel booking")
        print("6. Exit")
        print("=====")

        choice = input("Enter choice (1-6): ")

        match choice:
            case "1":
                display_flights(flights)
            case "2":
                dest = input("Enter destination city: ")

```

```

        search_flight(flights, dest)
    case "3":
        next_booking_id = book_flight(flights, bookings, next_booking_id)
    case "4":
        price = float(input("Enter single ticket price: "))
        count = int(input("Enter passenger count: "))
        total = calculate_price(price, count)
        print("Total Calculated Price: KSh " + str(total))
    case "5":
        cancel_booking(bookings)
    case "6":
        print("Thank you for using Airline Management System. Goodbye!")
        break
    case _:
        print("Invalid choice! Please select 1-6.")

if __name__ == "__main__":
    main_airline()

```

Concepts Practiced, Common Mistakes & Exercises

PYTHON CONCEPTS PRACTICED

Concept	CLI Calculator Usage	Airline System Usage
<code>while</code> Loops	Keeps main calculator menu open	Sustains system operation session
<code>match / case</code>	Routes arithmetic menu choices	Routes administrative menu actions
Functions & Returns	Performs discrete math calculations	Modularizes search, booking, pricing logic
2D Lists	Stores lists of numbers for sequence math	Stores tabular flight records and bookings

COMMON BEGINNER MISTAKES & DEBUGGING

- **TypeError concatenation:** Attempting `print("Total: " + total)` when `total` is a float. *Fix:* Explicitly convert using `str(total)`.
- **Unintended Integer Division:** Inputting integer parameters without converting to `float()` leads to unexpected precision loss in division operations.
- **Index Out of Range:** Querying `flight[4]` when a sublist only has 4 elements (indices 0 to 3).

10 CALCULATOR MODIFICATION EXERCISES

1. Add a option for percentage calculation (`(a / b) * 100`).
2. Add a feature to clear screen or reset state.
3. Modify average function to return median value.
4. Add square root calculation using exponent `a ** 0.5`.
5. Implement a operation counter that counts total calculations performed during session.
6. Add temperature conversion options (Celsius to Fahrenheit).
7. Add factor finding feature using modulus loop.
8. Validate number inputs to ensure non-negative values for specific operations.
9. Add currency conversion option using a fixed rate parameter.
10. Display calculation history by appending results to a history list.

10 AIRLINE SYSTEM MODIFICATION EXERCISES

1. Add a new flight route dynamically using `append()`.
2. Implement seating capacity limits for each flight.
3. Add return ticket calculation (multiply total fare by 1.8 for round trip).
4. Allow searching flights by departure origin city instead of destination.
5. Display total revenue earned from all active bookings.
6. Add passenger age validation (child discount under age 12).
7. Implement flight search by maximum budget threshold.
8. Print a detailed receipt summary formatted with borders.
9. Sort flights by price from lowest to highest using manual swap algorithm.
10. Add seat position choice (Window / Aisle) stored as additional element in booking record.

FUTURE IMPROVEMENTS (WHEN YOU LEARN ADVANCED CONCEPTS)

Advanced Concept	Calculator Enhancement (LATER)	Airline System Enhancement (LATER)
Dictionaries	Store memory registers (e.g. <code>{'M1': 42.0}</code>)	Represent flights as <code>{'code': 'KQ101', 'price': 8000}</code>
Exception Handling	Use <code>try/except ValueError</code> on user inputs	Catch invalid integer inputs during booking cleanly
File Handling & JSON	Save calculation history log to <code>history.txt</code>	Persist flights and bookings to <code>flights.json</code>
Object-Oriented (OOP)	Create <code>Calculator</code> class with state	Define <code>Flight</code> , <code>Passenger</code> , and <code>Booking</code> classes
Databases & APIs	Fetch live exchange rates from web APIs	Store bookings in SQLite relational database